



영수증 지출 관리 앱

Receipt Expense Tracker

프로그램 개요서 | Ver 1.0 | 2025

1. 프로젝트 개요

1.1 프로그램 목적

영수증(이미지/PDF)을 업로드하면 Upstage Vision LLM이 자동으로 내용을 인식하여 구조화된 지출 데이터로 변환하고, 사용자가 지출 내역을 손쉽게 조회·관리할 수 있는 웹 애플리케이션입니다.

1.2 핵심 가치

- 자동화: 영수증을 수동으로 입력하지 않고 AI가 자동 파싱
- 단순함: DB 없이 JSON 파일 기반으로 경량 운영
- 빠른 배포: Vercel + GitHub 연동으로 즉시 서비스 가능
- 확장성: FastAPI + LangChain 기반으로 기능 추가 용이

1.3 타겟 사용자

- 개인 지출 내역을 간편하게 관리하고 싶은 일반 사용자
- 복잡한 가계부 앱 없이 영수증만으로 소비 패턴을 파악하고 싶은 사용자
- AI 기반 OCR 기술을 활용한 사이드 프로젝트를 원하는 개발자

1.4 개발 조건 요약

항목	내용
개발 기간	1일 (단기 스프린트)
데이터베이스	미사용 (JSON 파일 기반 저장)

항목	내용
배포 환경	Vercel (프론트) + Vercel Serverless (백)
협업 도구	GitHub Repository
라이선스	개인 프로젝트

2. 주요 기능 목록

2.1 Must Have (반드시 구현)

#	기능	설명
M-01	영수증 이미지 업로드	JPG, PNG, PDF 파일 업로드 지원
M-02	OCR 자동 파싱	Upstage Vision LLM으로 영수증 내용 추출
M-03	구조화 JSON 출력	가게명, 날짜, 품목, 금액 자동 추출
M-04	지출 내역 목록 조회	저장된 지출 내역 카드형 리스트 표시
M-05	JSON 파일 저장	서버 로컬 JSON 파일에 데이터 누적 저장

2.2 Should Have (구현 권장)

#	기능	설명
S-01	지출 합계 대시보드	총 지출 금액, 품목별 합계 요약 표시
S-02	날짜 필터링	특정 기간의 지출 내역만 조회
S-03	지출 항목 삭제	잘못 등록된 영수증 데이터 삭제
S-04	파싱 결과 수정	OCR 결과를 사용자가 직접 수정 후 저장

2.3 Nice to Have (여유 시 구현)

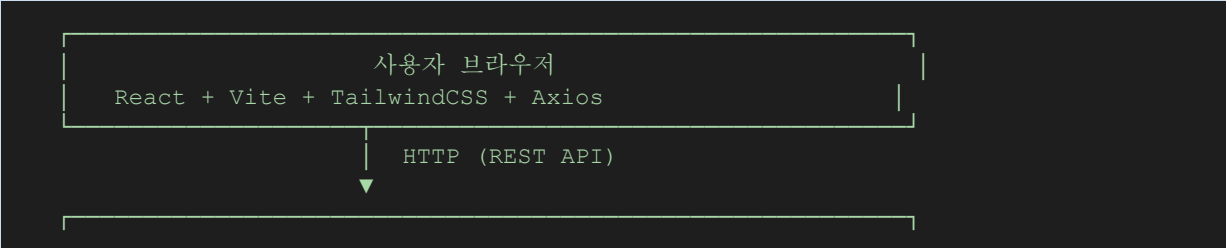
#	기능	설명
N-01	카테고리 자동 분류	식비, 교통, 쇼핑 등 LLM 기반 자동 분류
N-02	CSV 내보내기	지출 내역 전체를 CSV 파일로 다운로드
N-03	월별 차트	월별 지출 추이 시각화 (Chart.js)

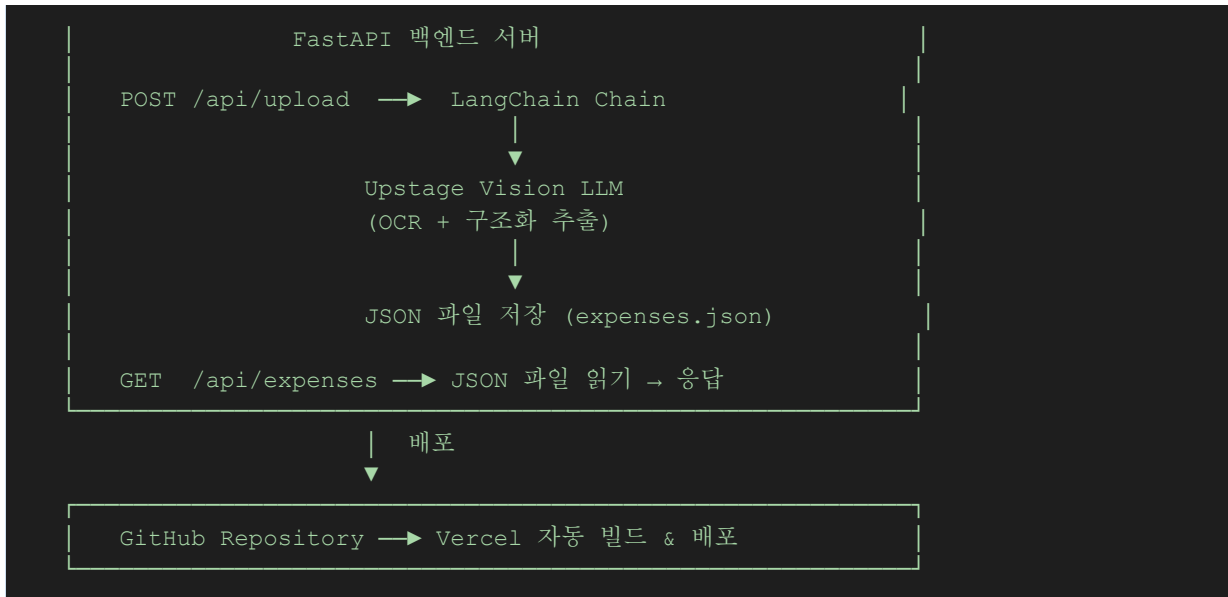
3. 기술 스택

구분	기술	버전/비고
프론트엔드 프레임워크	ReactJS	v18+
빌드 도구	Vite	v5+
스타일링	TailwindCSS	v3+
HTTP 통신	Axios	v1+
백엔드 프레임워크	Python FastAPI	v0.111+
AI/LLM 오케스트레이션	LangChain	v0.2+
OCR LLM	Upstage Document AI	document-digitization-vision
파일 처리	Pillow / pdf2image	이미지 전처리용
데이터 저장	JSON 파일 (서버 로컬)	DB 미사용
버전 관리	GitHub	main 브랜치 기준 배포
배포 플랫폼	Vercel	프론트 + 백엔드 서버리스
환경변수 관리	Vercel Environment Variables	UPSTAGE_API_KEY 등

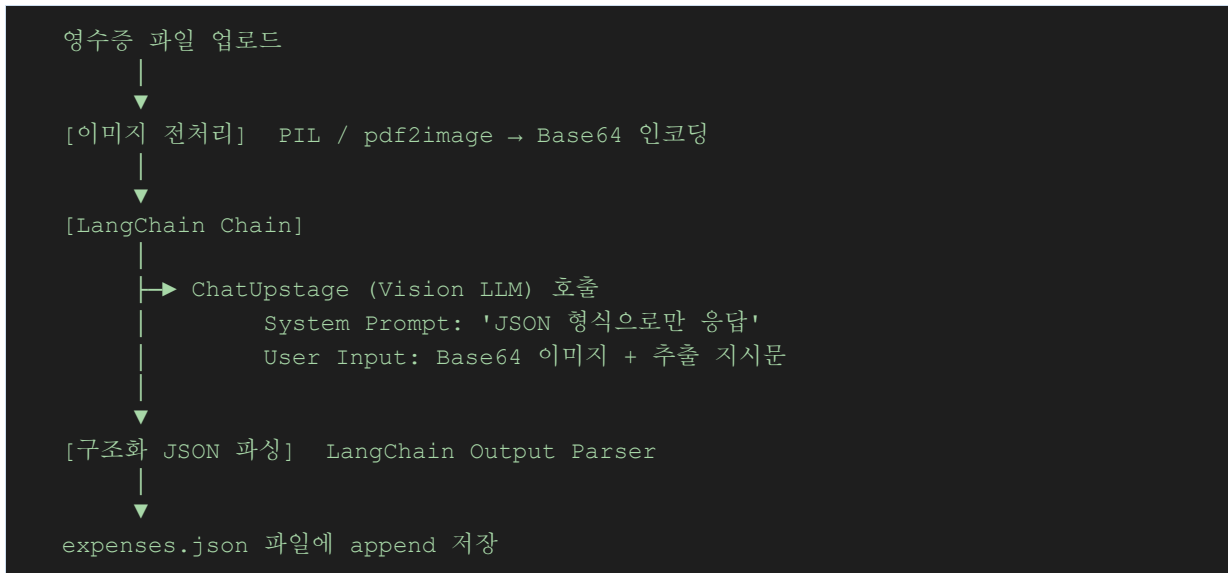
4. 시스템 아키텍처

4.1 전체 흐름





4.2 LangChain 처리 흐름



5. API 설계 (엔드포인트)

메서드	URL	설명	요청	응답
POST	/api/upload	영수증 업로드 및 OCR 파싱	multipart/form-data (file)	파싱된 JSON 객체
GET	/api/expenses	전체 지출 내역 조회	없음 (쿼리: ?from&to)	지출 목록 배열

메서드	URL	설명	요청	응답
DELETE	/api/expenses/{id}	특정 지출 내역 삭제	Path param: id	삭제 성공 메시지
PUT	/api/expenses/{id}	지출 내역 수정	JSON body	수정된 객체
GET	/api/summary	지출 합계 통계 조회	없음 (쿼리: ?month)	합계, 카테고리별 통계

6. 데이터 구조 정의

6.1 영수증 파싱 JSON 스키마

```
{
  "id": "uuid-v4-string",
  "created_at": "2025-07-15T14:30:00Z",
  "store_name": "이마트 강남점",
  "receipt_date": "2025-07-15",
  "receipt_time": "13:25",
  "category": "식료품",
  "items": [
    {
      "name": "신라면 멀티팩",
      "quantity": 2,
      "unit_price": 4500,
      "total_price": 9000
    },
    {
      "name": "바나나우유",
      "quantity": 1,
      "unit_price": 1800,
      "total_price": 1800
    }
  ],
  "subtotal": 10800,
  "discount": 500,
  "tax": 0,
  "total_amount": 10300,
  "payment_method": "신용카드",
  "raw_image_path": "uploads/receipt_20250715_001.jpg"
}
```

6.2 데이터 저장 방식 (DB 미사용)

□ DB 미사용 데이터 처리 전략

서버 측: expenses.json 파일에 배열 형태로 누적 저장 (append 방식)

클라이언트 측: 로컬 상태(React useState)로 실시간 UI 반영

영속성 보장: Vercel 배포 시 /tmp 디렉토리 활용 (또는 Vercel KV 옵션 고려)

□ 주의: Vercel 서버리스는 재시작 시 파일이 초기화될 수 있으므로, MVP 단계에서는 localStorage 병행 사용 권장

7. 화면 구성 (UI 흐름)

7.1 페이지 구성

페이지	경로	주요 컴포넌트	설명
메인 대시보드	/	SummaryCard, ExpenseList	지출 합계 + 내역 목록
업로드	/upload	DropZone, ParsePreview	영수증 업로드 및 파싱 결과 확인
상세 보기	/expense/:id	ExpenseDetail, EditForm	개별 지출 상세 + 수정

7.2 UI 컴포넌트 트리

```
App
├── Layout
│   ├── Header (로고 + 네비게이션)
│   └── Footer
├── pages/
│   ├── Dashboard
│   │   ├── SummaryCard (총 지출, 이번달 지출)
│   │   ├── FilterBar (날짜 범위 선택)
│   │   └── ExpenseList
│   │       └── ExpenseCard (영수증 카드 하나)
│   ├── UploadPage
│   │   ├── DropZone (드래그 앤 드롭 업로드)
│   │   ├── ProgressBar (OCR 처리 중 표시)
│   │   └── ParsePreview (파싱 결과 미리보기 + 저장 버튼)
│   └── ExpenseDetail
│       ├── ReceiptImage (원본 이미지 미리보기)
│       └── EditForm (수정 폼 + 삭제 버튼)
└── components/
    ├── Badge (카테고리 뱃지)
    ├── Modal (확인/취소 다이얼로그)
    └── Toast (저장/삭제 알림)
```

8. 1 일 개발 일정

시간	작업	산출물	우선도
09:00 ~ 10:00	환경 설정 & 프로젝트 초기화	FastAPI 뼈대, Vite+React 초기화, GitHub repo 생성	Must
10:00 ~ 11:30	백엔드 OCR API 개발	POST /api/upload → Upstage 연동 → JSON 반환	Must
11:30 ~ 12:30	데이터 저장/조회 API 개발	GET /api/expenses, DELETE, PUT 엔드포인트	Must
12:30 ~ 13:30	점심 & 휴식	-	-
13:30 ~ 15:30	프론트 업로드 화면 구현	DropZone, ParsePreview 컴포넌트 완성	Must
15:30 ~ 17:00	프론트 대시보드 구현	ExpenseList, SummaryCard 컴포넌트 완성	Must
17:00 ~ 18:00	Vercel 배포 설정	GitHub 연동, 환경변수 설정, 첫 배포 성공	Must
18:00 ~ 19:00	버그 수정 & 완성도 개선	전체 E2E 테스트, UI polish	Should
19:00 ~ 20:00	(여유) 필터 / 차트 추가	날짜 필터 or 월별 차트	Nice

9. 배포 전략

9.1 프로젝트 디렉토리 구조

```

receipt-tracker/
├── frontend/                # React + Vite
│   ├── src/
│   │   ├── pages/
│   │   ├── components/
│   │   └── api/             # Axios 인스턴스
│   ├── package.json
│   └── vite.config.js
├── backend/                 # Python FastAPI
│   ├── main.py
│   ├── routers/
│   ├── services/           # LangChain + Upstage 로직
│   ├── data/
│   │   └── expenses.json
│   └── requirements.txt
├── vercel.json              # 라우팅 설정
└── README.md

```

9.2 Vercel 배포 설정

```
// vercel.json
{
  "builds": [
    { "src": "frontend/package.json", "use": "@vercel/static-build" },
    { "src": "backend/main.py", "use": "@vercel/python" }
  ],
  "routes": [
    { "src": "/api/(.*)", "dest": "backend/main.py" },
    { "src": "/(.*)", "dest": "frontend/dist/$1" }
  ]
}
```

9.3 환경 변수

변수명	설명	설정 위치
UPSTAGE_API_KEY	Upstage API 인증 키	Vercel Environment Variables
VITE_API_BASE_URL	백엔드 API 기본 URL	Vercel (프론트 빌드 시 주입)
DATA_FILE_PATH	expenses.json 저장 경로	Vercel (백엔드 서버리스)

10. 제약사항 및 가정

10.1 DB 미사용에 따른 제약

□ 데이터 영속성 주의사항

Vercel 서버리스 함수는 요청마다 새로운 컨테이너에서 실행되어 파일 시스템이 유지되지 않을 수 있음

MVP 해결책 A: 클라이언트 localStorage에 데이터를 병행 저장하여 프론트에서 영속성 유지

MVP 해결책 B: 백엔드를 Railway, Render 등 일반 서버에 배포하여 파일 시스템 유지

장기적 확장: Vercel KV (Redis) 또는 Supabase (무료 플랜) 도입 시 DB 전환 권장

10.2 기타 가정 및 제약

- Upstage API Key 는 개인 발급하여 Vercel 환경 변수에 등록한다고 가정
- 영수증 이미지는 JPG, PNG, PDF 만 지원 (최대 10MB)
- 다국어 영수증 지원은 1 차 범위에서 제외 (한국어/영어 영수증만 지원)
- 동시 접속 사용자 수는 1 인 기준 (개인 프로젝트)
- 보안 인증(로그인)은 1 일 범위에서 제외하며, API URL 비공개로 최소 보안 유지